

Incidental Design II

Timeline and Postmortem Generation Features

Design Scope

This section extends the Design I timeline and postmortem feature set by introducing the user interfaces and major implementation logic for six connected use cases: viewing the incident timeline, filtering events, adding a timeline entry, editing an entry with revision history preservation, generating a postmortem draft, and creating and publishing corrective action items.

1. UI Design

1.1. Screen 1: Incident Timeline Dashboard

Purpose

Allows a user to review all events for a selected incident, inspect severity and state at a glance, and filter timeline entries by event type before starting postmortem work.

Incident Timeline			Incident #2481			
Title: Payment API Outage			Severity: Critical	State: Resolved		
Filters	Detection	Mitigation	Escalation	Recovery	Apply	Clear
09:03	Detection	Checkout alarms triggered for elevated 5xx rate.			System	
09:07	Escalation	Primary responder paged and incident declared.			A. Patel	
09:18	Mitigation	Traffic shifted away from failing deployment.			J. Kim	
09:39	Recovery	Error rate returned to baseline.			System	
10:05	Update	Customer communication posted and monitoring continued.			R. Henson	
Add Timeline Entry		View Postmortem		Export		

Primary inputs: selected incident, event-type filters.

Primary outputs: ordered timeline entries, filtered timeline view, quick access to postmortem workflow.

1.2. Screen 2: Add Timeline Entry Modal

Add Timeline Entry

Incident #2481 Payment API Outage

Event Type Select type

Description What happened, who acted, and what changed.

Timestamp Auto-generated

Validation: description required, event type required

Cancel Submit

Primary inputs: event type, description.

Primary outputs: new timeline event persisted and displayed in the dashboard.

1.3. Screen 3: Edit Timeline Entry and Revision History

Edit Timeline Entry

Current Entry
Event ID: 8742
Timestamp: 09:18
Type: Mitigation
Author: J. Kim
Description
Traffic shifted away from failing deployment after gateway rollback was initiated.
Save Cancel

Revision History
09:22
first cleanup edit.
09:30
Added deployment rollback.
09:44
Clarified traffic shift scope.
Compare Revision

Primary inputs: updated description text.

Primary outputs: revised timeline description plus an immutable revision record.

1.4. Screen 4: Postmortem Draft Workspace

Postmortem Draft

Status: Draft

Incident Summary: Payment API Outage | Critical | Resolved | 09:03–10:05

Root Cause

Impact

Resolution

Corrective Actions

Timeline Summary

09:03 detection from automated alerts. 09:07 escalation triggered. 09:18 mitigation started. 09:39 service recovered. 10:05 communication posted.

Generate Draft

Save

Approve

Publish

Create Action Item

Primary inputs: edited root cause, impact, resolution, and corrective action text.

Primary outputs: generated or saved draft, approval state, publication state.

1.5. Screen 5: Create Action Item Modal

Create Action Item

Description

Add synthetic health checks .

Assignee

Select user

Due Date

YYYY-MM-DD

Status

Open

Validation: description required, assignee required, due date must be after current date.

Cancel

Create

Primary inputs: description, assignee, due date.

Primary outputs: new corrective action item linked to the postmortem and incident.

2. Algorithm Design expressed with Pseudo-code

2.1. Algorithm 1: Filter Timeline by Event Type

Algorithm 1: FilterTimeline(incidentId, selectedTypes)

Input: incidentId, selectedTypes

Output: filteredEvents

```

1.1 incident  $\leftarrow$  FindIncidentById(incidentId);
1.2 if incident =  $\emptyset$  then
1.3   return error "Incident not found";
1.4 allEvents  $\leftarrow$  GetTimelineEventsForIncident(incidentId);
1.5 if selectedTypes is empty then
1.6   return SortByTimestamp(allEvents);
1.7 filteredEvents  $\leftarrow$  [ ];
1.8 foreach event in allEvents do
1.9   if event.eventType  $\in$  selectedTypes then
1.10    Append(filteredEvents, event);
1.11 filteredEvents  $\leftarrow$  SortByTimestamp(filteredEvents);
1.12 return filteredEvents;

```

2.2. Algorithm 2: Add Timeline Entry

Algorithm 2: AddTimelineEntry(userId, incidentId, eventType, description)

Input: userId, incidentId, eventType, description

Output: newEvent or error

```

2.1 user  $\leftarrow$  FindUserById(userId);
2.2 incident  $\leftarrow$  FindIncidentById(incidentId);
2.3 if user =  $\emptyset$  and incident =  $\emptyset$  then
2.4   return error "User and incident not found";
2.5 if user =  $\emptyset$  then
2.6   return error "User not found";
2.7 if incident =  $\emptyset$  then
2.8   return error "Incident not found";
2.9 if eventType is empty then
2.10   return error "Event type is required";
2.11 if Trim(description) = "" then
2.12   return error "Description is required";
2.13 if Length(description) > 500 then
2.14   return error "Description exceeds maximum length";
2.15 newEvent.eventId  $\leftarrow$  GenerateEventId();
2.16 newEvent.incidentId  $\leftarrow$  incidentId;
2.17 newEvent.userId  $\leftarrow$  userId;
2.18 newEvent.timestamp  $\leftarrow$  CurrentDateTime();
2.19 newEvent.eventType  $\leftarrow$  eventType;
2.20 newEvent.description  $\leftarrow$  description;
2.21 SaveTimelineEvent(newEvent);
2.22 return newEvent;

```

2.3. Algorithm 3: Edit Timeline Entry and Store Revision

Algorithm 3: EditTimelineEntry(userId, eventId, newDescription)

Input: userId, eventId, newDescription

Output: updatedEvent or error

```

3.1 user ← FindUserById(userId);
3.2 event ← FindTimelineEventById(eventId);
3.3 if user =  $\emptyset$  then
3.4   | return error “User not found”;
3.5 if event =  $\emptyset$  then
3.6   | return error “Timeline event not found”;
3.7 if Trim(newDescription) = “” then
3.8   | return error “Description is required”;
3.9 if newDescription = event.description then
3.10  | return error “No change detected”;
3.11 revision.revisionId ← GenerateRevisionId();
3.12 revision.eventId ← eventId;
3.13 revision.userId ← userId;
3.14 revision.editedAt ← CurrentDateTime();
3.15 revision.previousDescription ← event.description;
3.16 SaveRevision(revision);
3.17 event.description ← newDescription;
3.18 UpdateTimelineEvent(event);
3.19 return event;

```

2.4. Algorithm 4: Generate Postmortem Draft

Algorithm 4: GeneratePostmortemDraft(userId, incidentId)

Input: userId, incidentId

Output: postmortemDraft or error

```

4.1 incident  $\leftarrow$  FindIncidentById(incidentId);
4.2 user  $\leftarrow$  FindUserById(userId);
4.3 if incident =  $\emptyset$  then
4.4   return error "Incident not found";
4.5 if user =  $\emptyset$  then
4.6   return error "User not found";
4.7 timeline  $\leftarrow$  SortByTimestamp(GetTimelineEventsForIncident(incidentId));
4.8 if timeline is empty then
4.9   return error "Cannot generate postmortem without timeline data";
4.10 draft.postmortemId  $\leftarrow$  GeneratePostmortemId();
4.11 draft.incidentId  $\leftarrow$  incidentId;
4.12 draft.userId  $\leftarrow$  userId;
4.13 draft.status  $\leftarrow$  "Draft";
4.14 draft.impact  $\leftarrow$  SummarizeImpactFromTimeline(timeline);
4.15 draft.resolution  $\leftarrow$  SummarizeResolutionFromTimeline(timeline);
4.16 draft.rootCause  $\leftarrow$  "Pending root cause analysis";
4.17 draft.correctiveActions  $\leftarrow$  DeriveInitialCorrectiveActions(timeline);
4.18 SavePostmortem(draft);
4.19 return draft;

```

2.5. Algorithm 5: Create Action Item

Algorithm 5: CreateActionItem(userId, postmortemId, description, assigneeId, dueDate)

Input: userId, postmortemId, description, assigneeId, dueDate

Output: actionItem or error

```

5.1 creator  $\leftarrow$  FindUserById(userId);
5.2 postmortem  $\leftarrow$  FindPostmortemById(postmortemId);
5.3 assignee  $\leftarrow$  FindUserById(assigneeId);
5.4 if creator =  $\emptyset$  then
5.5   return error "Creator not found";
5.6 if postmortem =  $\emptyset$  then
5.7   return error "Postmortem not found";
5.8 if assignee =  $\emptyset$  then
5.9   return error "Assignee not found";
5.10 if Trim(description) = "" then
5.11   return error "Description is required";
5.12 if dueDate  $\leq$  CurrentDateTime() then
5.13   return error "Due date must be in the future";
5.14 item.actionItemId  $\leftarrow$  GenerateActionItemId();
5.15 item.postmortemId  $\leftarrow$  postmortemId;
5.16 item.incidentId  $\leftarrow$  postmortem.incidentId;
5.17 item.userId  $\leftarrow$  assigneeId;
5.18 item.description  $\leftarrow$  description;
5.19 item.status  $\leftarrow$  "Open";
5.20 item.dueDate  $\leftarrow$  dueDate;
5.21 SaveActionItem(item);
5.22 return item;

```

2.6. Algorithm 6: Approve and Publish Postmortem

Algorithm 6: PublishPostmortem(userId, postmortemId)

Input: userId, postmortemId

Output: publishedPostmortem or error

```

6.1 user ← FindUserById(userId);
6.2 postmortem ← FindPostmortemById(postmortemId);
6.3 if user = ∅ then
6.4   | return error “User not found”;
6.5 if postmortem = ∅ then
6.6   | return error “Postmortem not found”;
6.7 if Trim(postmortem.rootCause) = “” or Trim(postmortem.impact) = “” then
6.8   | return error “Required postmortem sections are incomplete”;
6.9 if Trim(postmortem.resolution) = “” or Trim(postmortem.correctiveActions) = “” then
6.10  | return error “Required postmortem sections are incomplete”;
6.11 if postmortem.status = “Published” then
6.12  | return error “Postmortem already published”;
6.13 if postmortem.status ≠ “Approved” then
6.14  | postmortem.status ← “Approved”;
6.15  | postmortem.approvedAt ← CurrentDateTime();
6.16 postmortem.status ← “Published”;
6.17 postmortem.publishedAt ← CurrentDateTime();
6.18 UpdatePostmortem(postmortem);
6.19 return postmortem;

```

3. Traceability Between UI and Algorithms

UI Artifact	Supported Logic
Incident Timeline Dashboard	Runs <code>FilterTimeline</code> to display all events or only the selected event categories.
Add Timeline Entry Modal	Runs <code>AddTimelineEntry</code> and shows validation errors before submission.
Edit Timeline Entry and Revision History	Runs <code>EditTimelineEntry</code> and then refreshes revision records associated with the edited event.
Postmortem Draft Workspace	Runs <code>GeneratePostmortemDraft</code> , supports editing required sections, and triggers publication workflow.
Create Action Item Modal	Runs <code>CreateActionItem</code> to attach follow-up work to a postmortem.
Publish Button on Postmortem Workspace	Runs <code>PublishPostmortem</code> and blocks publication if required sections are incomplete.

4. Implementation Notes

The algorithms intentionally abstract away lower-level persistence details such as SQL queries or ORM method calls while preserving the major program logic expected for this stage of design. Input validation is included for every UI that accepts user-entered data, revision history is preserved before timeline edits overwrite existing content, and publication enforces a valid postmortem state transition.